

PROJECT DOCUMENTATION

Course: Software Development Methodologies

Project: Design and Implementation of an Online Shopping Information System Using the Waterfall Methodology

1. General Information

Project Title: Online Shopping System

Authors: Dilnaz, Zhanara, Otabek, Hicham, Elif

Program/ Year: Software Development Methodologies L4

Semester: 4

Supervisor: Marcin Kacprowicz

Submission Date: 06.02.2026

Github Link : <https://github.com/HichamElOuajibi/OnlineShop>

2. Project Description

2.1. Project Objective

The Online Shopping System is a web-based platform that allows customers to browse, search, and purchase products online.

It includes customer functions such as registration, login, cart management, and order checkout, as well as administrative features for managing products and viewing orders.

The goal is to build a functional prototype following the Waterfall methodology, covering all key stages: requirements, design, implementation, testing, and documentation.

2.2. Project Scope

In-Scope Features

- User registration and authentication.
- Product browsing and search functionality.
- Shopping cart management.
- Order placement and checkout process.
- Administrative management of products and orders

Out-of-Scope Features

- Integration with real payment gateways (ex: PayPal, Stripe).
- Delivery tracking with external logistics systems.
- Development of a native mobile application.

3. System Requirements

3.1. Functional Requirements

ID	Requirements	Description
FR1	User Registration/Login	The system allows users to register an account and log in securely
FR2	Product Catalog	Displays available products with details such as name, image, and price.
FR3	Add to Cart	Users can add items to their cart and view the selected products
FR4	Checkout Process	Users can proceed to checkout and confirm their order.
FR5	Admin Management	Administrators can manage products and review customer orders.

3.2. Non-functional Requirements

ID	Requirements	Description
NFR1	Usability	The interface should be user-friendly and easy to navigate
NFR2	Performance	The system should respond to user requests time ≤ 2 seconds.
NFR3	Security	User credentials and data must be stored securely
NFR4	Scalability	The system architecture should allow future feature expansion
NFR5	Compatibility	The system should be compatible with major browsers, operating systems, and devices (desktop, tablet, mobile).
NFR6	Reliability	The system should handle errors without crashing or losing data.

4. Methodology Implementation: Waterfall Model

Reason for Choice:

The Waterfall Model was selected because it is simple, linear, and easy to manage for a small academic project.

Each stage follows a clear sequence, making documentation and progress tracking straightforward.

This approach is suitable when the requirements are well-defined and unlikely to change frequently.

The development process followed the five main stages of the Waterfall Model:

1. Requirement Analysis:

All functional and non-functional requirements were gathered and documented, defining the main features such as user registration, product catalog, and checkout process.

2. System Design:

Based on the requirements, the system architecture, interface layout, and database structure were designed to support all planned functionalities.

3. Implementation (Development):

The system was developed according to the design specifications. Each module (e.g., login, catalog, cart) was coded sequentially following the defined plan.

4. Testing:

After development, the system was tested to identify and fix any errors. Functional and usability testing were performed to ensure that all requirements were met

5. Deployment and Maintenance:

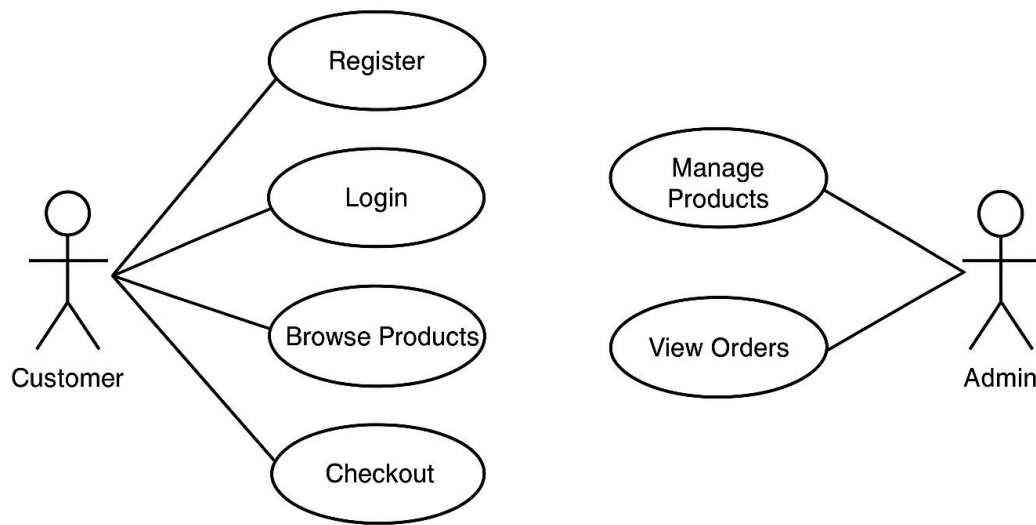
The final version of the system was deployed for review. Minor adjustments were made based on feedback to improve performance and usability.

5. UML Diagrams

5.1. Use Case Diagram

Insert the UML diagram showing user interactions with the system.

Online Shopping System



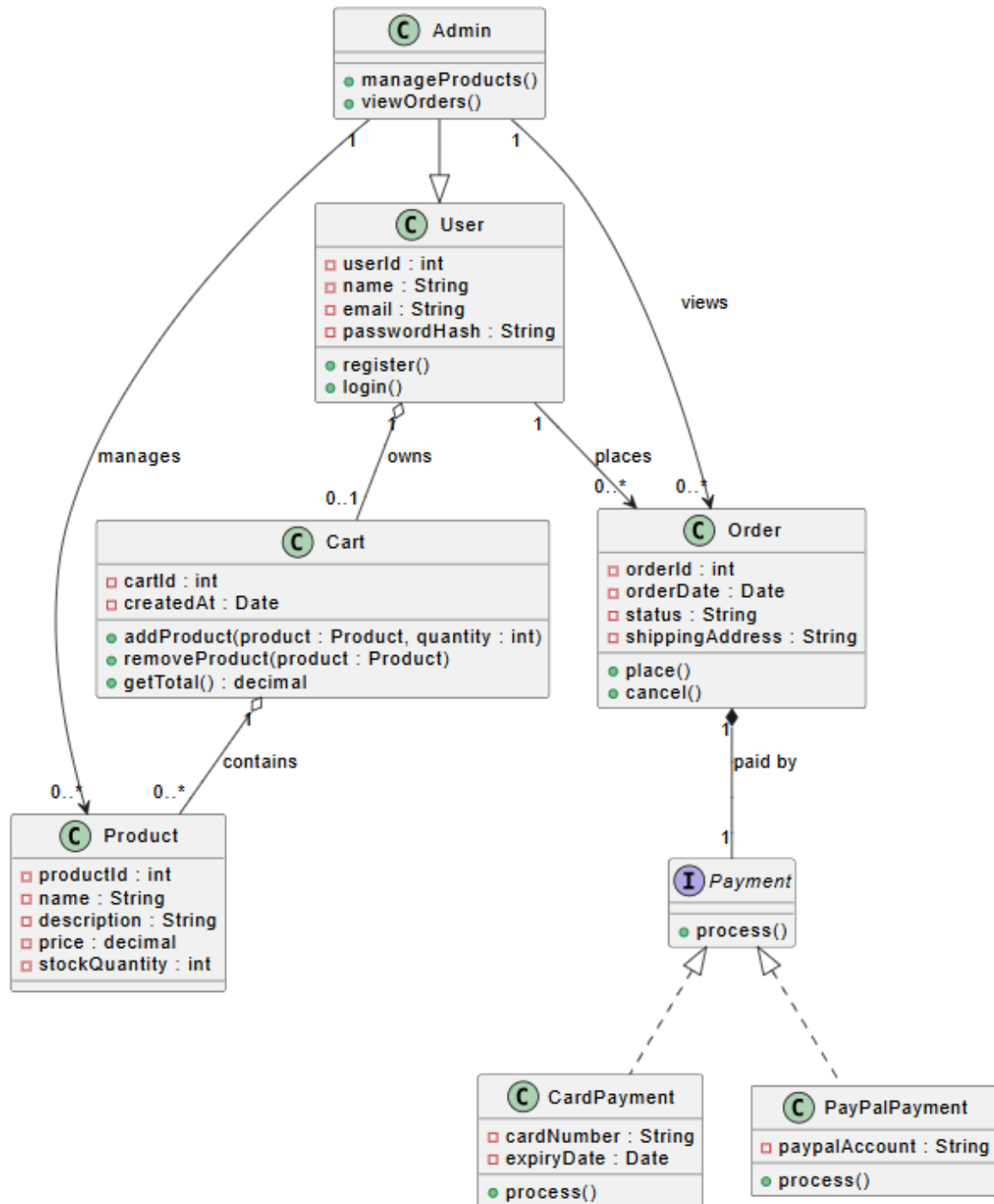
Customer can:

- Register
- Login
- Browse Products
- Checkout

Admin can:

- Manage Products
- View Orders

5.2. Class Diagram

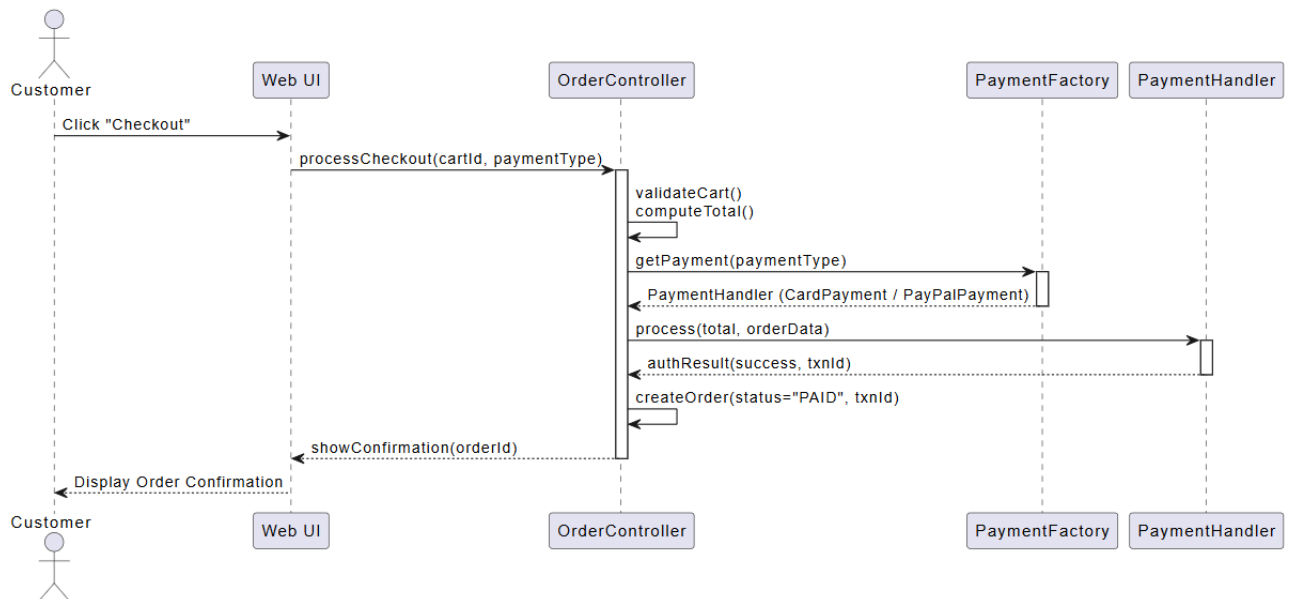


Class	Description
User	Represents a customer of the system. Stores personal info (name, email, password) and provides actions such as registering and logging in.
Admin	A specialized type of User with additional responsibilities. Can manage products and view system orders.
Product	Represents an item available for purchase. Contains the product's name, description, price, and stock level.
Cart	Holds the items a user intends to purchase. Allows adding/removing products and calculating the cart total.
Order	Represents a completed purchase, storing date, order status, and shipping information.
Payment	Defines the common operation for executing a payment. Used as a base abstraction for different payment types.
Card Payment	Implements the Payment interface. Processes payments using card details such as card number and expiry date.
PayPal Payment	Implements the Payment interface. Processes payments using a PayPal account.

Relationships

- **User → Cart**
A user can have at most one active cart.
- **Cart → Product**
A cart can contain multiple products.
- **User → Order**
A user can place multiple orders.
- **Order → Payment**
Each order has exactly one payment method used to complete it.
- **Payment → CardPayment / PayPalPayment**
These classes implement the Payment interface.
- **Admin → Product / Order**
An admin manages product catalog operations and can view all orders.
- **Admin → User**
Admin is a specialization of User (inheritance).

5.3. Sequence Diagram



Actors & Objects

- **Customer** → The user who initiates the checkout process.
- **Web UI** → The interface the customer interacts with (webpage/mobile screen).
- **OrderController** → Handles checkout logic and coordinates the process.
- **PaymentFactory** → Creates the correct payment method object based on user selection (Card or PayPal).
- **Payment (CardPayment / PayPalPayment)** → The specific payment processor that executes the transaction.

Flow Explanation (Messages)

- **Customer → Web UI:** Starts the checkout process.
- **Web UI → OrderController:** Calls processCheckout() with cart details and selected payment type.
- **OrderController → PaymentFactory:** Requests a payment object by calling getPayment(paymentType).
- **PaymentFactory → OrderController:** Returns the correct Payment implementation (e.g., CardPayment, PayPalPayment).
- **OrderController → Payment:** Executes the payment by calling process(amount).
- **Payment → OrderController:** Returns the payment result/status.
- **OrderController → Web UI:** Sends order confirmation details.
- **Web UI → Customer:** Displays the "Order Confirmation" screen.

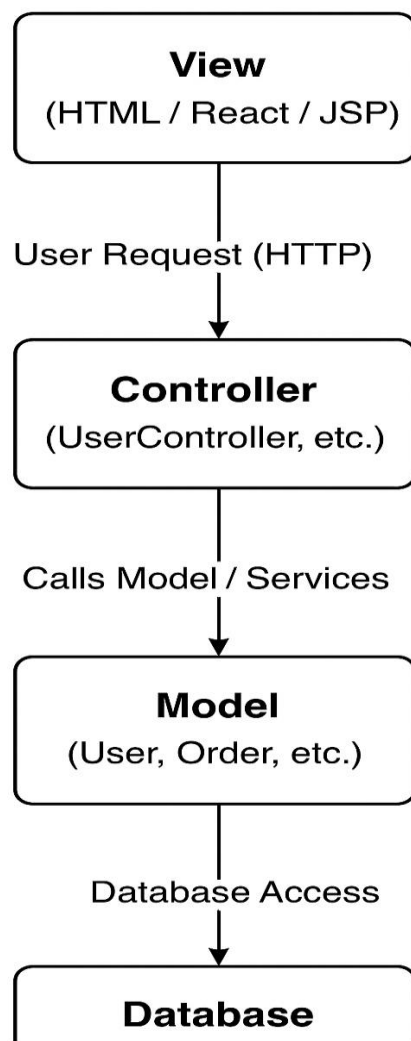
6. Design and Architectural Patterns

6.1. Architecture Diagram (MVC)

Model-View-Controller pattern:

The system is designed using the **Model-View-Controller (MVC)** pattern, which separates the application into three main layers to improve modularity and maintainability.

- **Model** – Represents the data and business logic of the online shopping system.
Examples: User, Product, Cart, Order.
- **View** – Responsible for displaying information to the user and collecting input.
Examples: product catalog page, cart page, checkout page, login/register forms.
- **Controller** – Handles user requests, coordinates interactions between Views and Models, and returns the appropriate output.
Examples: UserController, ProductController, OrderController.



Flow of Data

- The View sends user input (e.g., login, add to cart, browse product).
- The Controller receives the request and invokes the necessary Model logic.
- The Model processes the data and returns results.
- The Controller sends the results back to the View for display.

Why MVC is used?

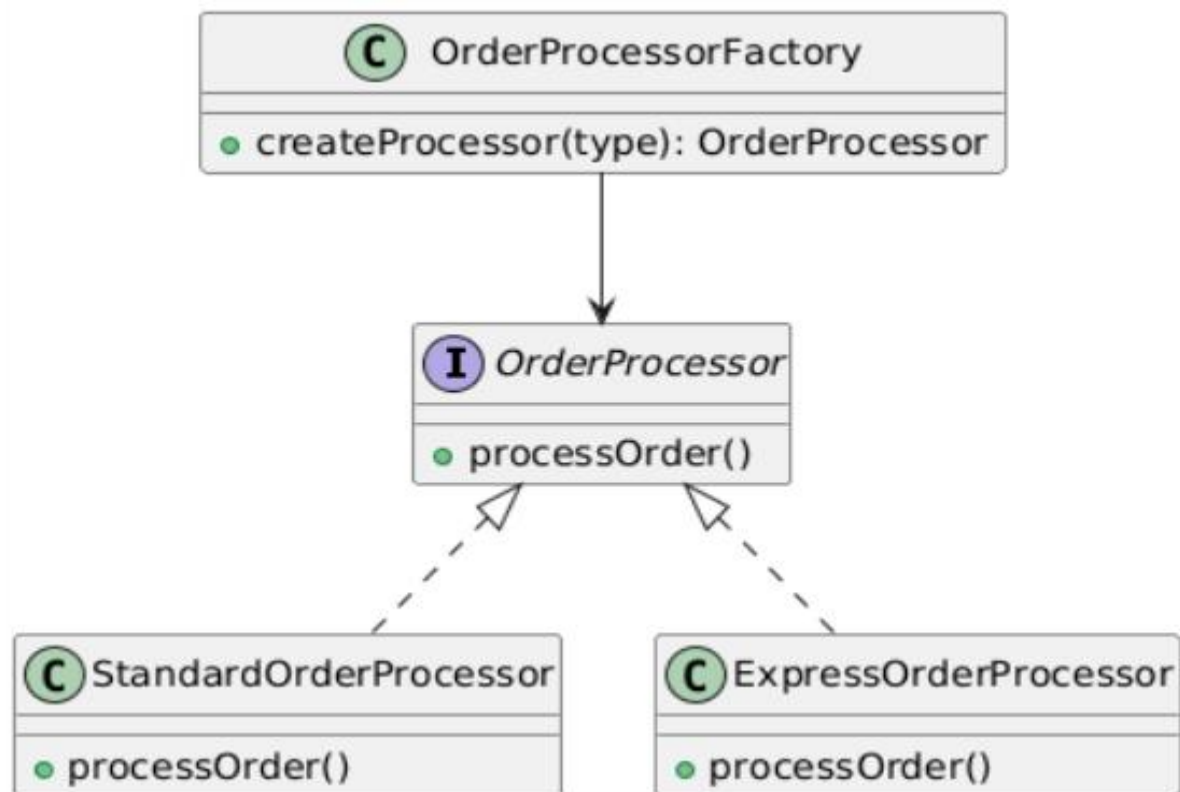
- Clear separation between interface, logic, and data
- Easy to maintain and extend
- Supports team collaboration
- Follows standard design practices for web systems

How MVC is used in this system?

- UserController manages authentication operations
- ProductController handles product browsing and search
- OrderController manages order placement and cart operations
- Models store and validate business data
- Views display system pages to the user

6.2 Design Pattern – Factory Pattern

The **Factory Pattern** is used to centralize and simplify the creation of certain objects that may vary based on system actions. Instead of creating objects directly inside controllers, the Factory is responsible for generating the correct type of object.



Purpose of the Factory Pattern

- Encapsulates object creation logic in one place
- Reduces coupling between components
- Makes the system easier to extend with new object types
- Improves readability and maintainability

How the Factory Pattern is used in the system?

In parts of the system where different processing classes may be needed (for example: different types of order handlers, or different types of product processors), the Factory returns the appropriate object without exposing creation details.

Benefits of using the Factory Pattern

- New types can be added without modifying existing controllers
- Improves testability and maintainability

- Keeps object creation centralized and organized
- Reduces repeated or duplicated logic.

7. Testing

Testing Strategy

The project uses JUnit 5 for unit and web-layer testing. Tests are located in src/test/java and cover both service logic and controller endpoints. Controller tests use Spring Boot @WebMvcTest with MockMvc to simulate HTTP requests without running the full server. Service tests call methods directly to validate business rules



7.2. Tools and FrameWork

JUnit 5 for unit testing

MockMvc for controller/API testing

Mockito for mocking service dependencies

7.3. Test Cases and Results

Test Id	Description	Result
T1	User registration + login (UserServiceTest)	Passed
T2	Cart validation (invalid quantity/product)	Passed
T3	Checkout creates order + clears cart	Passed
T4	Admin API endpoints (CRUD + orders)	Passed
T5	Application context loads	Passed

7.4. Test Execution

Tests were run with the following command in the project root:

.\mvnw.cmd test

```
2026-02-06T10:26:59.594+01:00 INFO 28936 --- [OnlineShop] [ main] c.c.o.OnlineShopApplicationTests : Starting OnlineShopApplicationTests using Java 25.0.1 with PID 28936 (st
/Users/vac/OnlineShop)
2026-02-06T10:26:59.594+01:00 INFO 28936 --- [OnlineShop] [ main] c.c.o.OnlineShopApplicationTests : No active profile set, falling back to 1 default profile: "default"
2026-02-06T10:26:59.963+01:00 INFO 28936 --- [OnlineShop] [ main] c.c.o.OnlineShopApplicationTests : Started OnlineShopApplicationTests in 0.39 seconds (process running for
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.408 s -- in com.codeonlineshop.onlineShop.OnlineShopApplicationTests
[INFO] Running com.codeonlineshop.onlineShop.OrdersServiceCheckoutTest
Paypal Payment Service
Amount: 1700.0$
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.005 s -- in com.codeonlineshop.onlineShop.OrdersServiceCheckoutTest
[INFO] Running com.codeonlineshop.onlineShop.OrdersServiceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.885 s -- in com.codeonlineshop.onlineShop.OrdersServiceTest
[INFO] Running com.codeonlineshop.onlineShop.PaypalPaymentServiceTest
Paypal Payment Service
Amount: 50.0$
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.002 s -- in com.codeonlineshop.onlineShop.PaypalPaymentServiceTest
[INFO] Running com.codeonlineshop.onlineShop.StripePaymentServiceTest
Stripe Payment Service
Amount: 75.0$
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.002 s -- in com.codeonlineshop.onlineShop.StripePaymentServiceTest
[INFO] Running com.codeonlineshop.onlineShop.UserServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.006 s -- in com.codeonlineshop.onlineShop.UserServiceTest
[INFO] Results:
[INFO] Tests run: 17, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 5.992 s
[INFO] Finished at: 2026-02-06T10:27:00+01:00
[INFO]
```

```
Spring
:: Spring Boot :: (v3.5.7)
2026-02-06T10:26:59.272+01:00 INFO 28936 --- [OnlineShop] [ main] c.c.onlineShop.CartControllerTest : Starting CartControllerTest using Java 25.0.1 with PID 28936 (started by vac in C:\Users\
vac\OnlineShop)
2026-02-06T10:26:59.272+01:00 INFO 28936 --- [OnlineShop] [ main] c.c.onlineShop.CartControllerTest : No active profile set, falling back to 1 default profile: "default"
2026-02-06T10:26:59.582+01:00 INFO 28936 --- [OnlineShop] [ main] o.s.b.t.e.w.SpringBootMockServletContext : Initializing Spring TestDispatcherServlet ''
2026-02-06T10:26:59.583+01:00 INFO 28936 --- [OnlineShop] [ main] o.s.t.web.servlet.TestDispatcherServlet : Initializing Servlet ''
2026-02-06T10:26:59.583+01:00 INFO 28936 --- [OnlineShop] [ main] o.s.t.web.servlet.TestDispatcherServlet : Completed initialization in 0 ms
2026-02-06T10:26:59.588+01:00 INFO 28936 --- [OnlineShop] [ main] c.c.onlineShop.CartControllerTest : Started CartControllerTest in 0.269 seconds (process running for 3.544)
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.332 s -- in com.codeonlineshop.onlineShop.CartControllerTest
[INFO] Running com.codeonlineshop.onlineShop.CartServiceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.013 s -- in com.codeonlineshop.onlineShop.CartServiceTest
[INFO] Running com.codeonlineshop.onlineShop.OnlineShopApplicationTests
2026-02-06T10:26:59.584+01:00 INFO 28936 --- [OnlineShop] [ main] t.c.s.AnnotationConfigContextLoaderUtils : Could not detect default configuration classes for test class [com.codeonlineshop.online
Shop.OnlineShopApplicationTests]: OnlineShopApplicationTests does not declare any static, non-private, non-final, nested classes annotated with @Configuration.
2026-02-06T10:26:59.587+01:00 INFO 28936 --- [OnlineShop] [ main] b.t.c.SpringBootTestContextBootstrapper : Found @SpringBootConfiguration com.codeonlineshop.onlineShop.OnlineShopApplication for te
st class com.codeonlineshop.onlineShop.OnlineShopApplicationTests
2026-02-06T10:26:59.573+01:00 INFO 28936 --- [OnlineShop] [ main] o.s.b.d.r.RestartApplicationListener : Restart disabled due to context in which it is running
```

```
Spring
:: Spring Boot :: (v3.5.7)
2026-02-06T10:26:59.279+01:00 INFO 28936 --- [OnlineShop] [ main] c.c.onlineShop.AdminControllerTest : Starting AdminControllerTest using Java 25.0.1 with PID 28936 (started by vac in C:\Users\
vac\OnlineShop)
2026-02-06T10:26:59.279+01:00 INFO 28936 --- [OnlineShop] [ main] c.c.onlineShop.AdminControllerTest : No active profile set, falling back to 1 default profile: "default"
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JVM. Please add Mockito to your build as described in Mockito's documentation
: https://java.dock/16/doc/org.mockito/mockito-core/latest/arg-mockito/org.mockito/mockito.html#3
WARNING: A Java agent has been loaded dynamically (C:\Users\vac\2\repository\jnetbyteduddy\byte-buddy-agent\1.17.0\byte-buddy-agent-1.17.0.jar)
WARNING: If a serviceability tool is in use, please run with -XX:EnableDynamicAgentLoading to hide this warning
WARNING: Dynamic loading of agents will be disabled by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
2026-02-06T10:26:58.831+01:00 INFO 28936 --- [OnlineShop] [ main] o.s.b.t.e.w.SpringBootMockServletContext : Initializing Spring TestDispatcherServlet ''
2026-02-06T10:26:58.832+01:00 INFO 28936 --- [OnlineShop] [ main] o.s.t.web.servlet.TestDispatcherServlet : Initializing Servlet ''
2026-02-06T10:26:58.834+01:00 INFO 28936 --- [OnlineShop] [ main] o.s.t.web.servlet.TestDispatcherServlet : Completed initialization in 1 ms
2026-02-06T10:26:58.862+01:00 INFO 28936 --- [OnlineShop] [ main] c.c.onlineShop.AdminControllerTest : Started AdminControllerTest in 1.932 seconds (process running for 2.897)
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.659 s -- in com.codeonlineshop.onlineShop.AdminControllerTest
[INFO] Running com.codeonlineshop.onlineShop.CartControllerTest
2026-02-06T10:26:59.217+01:00 INFO 28936 --- [OnlineShop] [ main] t.c.s.AnnotationConfigContextLoaderUtils : Could not detect default configuration classes for test class [com.codeonlineshop.online
Shop.CartControllerTest]: CartControllerTest does not declare any static, non-private, non-final, nested classes annotated with @Configuration.
2026-02-06T10:26:59.222+01:00 INFO 28936 --- [OnlineShop] [ main] b.t.c.SpringBootTestContextBootstrapper : Found @SpringBootConfiguration com.codeonlineshop.onlineShop.OnlineShopApplication for te
st class com.codeonlineshop.onlineShop.CartControllerTest
2026-02-06T10:26:59.239+01:00 INFO 28936 --- [OnlineShop] [ main] o.s.b.d.r.RestartApplicationListener : Restart disabled due to context in which it is running
```

```
PS C:\Users\vac\OnlineShop> mvnw.cmd test
WARNING: A potentially deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::staticFieldBase has been called by com.google.inject.internal.aop.HiddenClassDefiner (file:/C:/Users/vac/.m2/wrapper/dists/apache-maven-3.9.9/apache-maven-3.9.9-bin.zip!/bin/ma
vew-3.9.9.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.inject.internal.aop.HiddenClassDefiner
WARNING: sun.misc.Unsafe::staticFieldBase will be removed in a future release
[INFO] Scanning for projects...
[INFO]
[INFO] --- com.codeonlineshop.OnlineShop ---
[INFO] Building OnlineShop 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ OnlineShop ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ OnlineShop ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ OnlineShop ---
[INFO] skip non existing resourceDirectory C:\Users\vac\OnlineShop\src\testResources
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ OnlineShop ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- surefire:3.0.0-M5:test (default-test) @ OnlineShop ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] ----- T E S T S -----
[INFO] Running com.codeonlineshop.onlineShop.AdminControllerTest
10:26:56.692 [main] INFO org.springframework.test.context.support.AnnotationConfigContextLoaderUtils : Could not detect default configuration classes for test
class [com.codeonlineshop.onlineShop.AdminControllerTest]: AdminControllerTest does not declare any static, non-private, non-final, nested classes annotated with @Configuration.
10:26:56.809 [main] INFO org.springframework.boot.test.context.SpringBootTestContextBootstrapper : Found @SpringBootConfiguration com.codeonlineshop.onlineShop.OnlineShopApplication for te
st class com.codeonlineshop.onlineShop.AdminControllerTest
10:26:56.904 [main] INFO org.springframework.boot.devtools.restart.RestartApplicationListener : Restart disabled due to context in which it is running
```

7.5. Example Test Explanation

Example: UserServiceTest

Registers a user

Logs in with the same credentials

Checks that userId, username, and role are correct

Also tests invalid password case

```
package com.codeonlineshop.onlineshop;

import com.codeonlineshop.onlineshop.dto.LoginRequest;
import com.codeonlineshop.onlineshop.dto.RegisterRequest;
import com.codeonlineshop.onlineshop.model.Role;
import com.codeonlineshop.onlineshop.model.User;
import org.junit.jupiter.api.Test;
import org.springframework.web.server.ResponseStatusException;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

class UserServiceTest {

    @Test
    void registerAndLogin_shouldReturnUser() {
        UserService userService = new UserService();
        RegisterRequest registerRequest = new RegisterRequest("alice", "password123", "CUSTOMER");

        User registered = userService.register(registerRequest);
        LoginRequest loginRequest = new LoginRequest("alice", "password123");
        User loggedIn = userService.login(loginRequest);

        assertEquals(registered.getId(), loggedIn.getId());
        assertEquals("alice", loggedIn.getUsername());
        assertEquals(Role.CUSTOMER, loggedIn.getRole());
    }

    @Test
    void login_withWrongPassword_shouldFail() {
        UserService userService = new UserService();
        RegisterRequest registerRequest = new RegisterRequest("bob", "secret", "CUSTOMER");
        userService.register(registerRequest);

        LoginRequest loginRequest = new LoginRequest("bob", "wrong");

        assertThrows(ResponseStatusException.class, () -> userService.login(loginRequest));
    }
}
```

8. System Architecture

The **Online Shop Information System** was developed using the **Spring Boot 3.5.7 framework** on **Java 25**, following a layered **Model-View-Controller (MVC)** architecture and modern software engineering principles. The system's architecture separates concerns into distinct modules to ensure high cohesion and low coupling.

8.1 Architectural Overview

The application consists of three main layers:

1. **Presentation Layer (Controller)**

Handles incoming HTTP requests, maps routes to controller methods, and returns responses or HTML views.

Implemented using Spring MVC annotations such as `@Controller` and `@RequestMapping`.

Example: `HomeController` serves web pages and routes requests to appropriate services.

2. **Business Layer (Service)**

Contains core business logic and enforces application rules.

Services such as `OrderService`, `StripePaymentService`, and `PaypalPaymentService` are annotated with `@Service` and managed by the Spring IoC Container.

This layer also applies the **Strategy Pattern** and **Dependency Injection** to enable flexible switching between payment providers.

3. **Data / Integration Layer**

Currently, this layer manages simulated data and service integration points (e.g., payment gateways).

Future enhancements will include database persistence via Spring Data JPA.

8.2 Component Interaction

1. The **user** initiates an action (e.g., checkout) through the web interface.
2. The **Controller** receives the request and delegates the business logic to the **OrderService**.
3. The **OrderService** obtains an appropriate **PaymentService** implementation (Stripe or PayPal) through **Spring's Dependency Injection** mechanism.
4. The **PaymentService** processes the payment, and the **OrderService** finalizes the order.
5. The response is returned to the user via the **Controller**, completing the MVC cycle.

8.3 Design Patterns and Principles

- **MVC Pattern:** Separates presentation, logic, and data access for modularity.
- **Strategy Pattern:** Implements multiple payment strategies (StripePaymentService, PaypalPaymentService) under a shared interface PaymentService.
- **Dependency Injection:** Achieves loose coupling between classes by delegating object creation to the Spring IoC container.
- **SOLID Principles:** Applied, particularly *Dependency Inversion* and *Open/Closed* principles, ensuring flexibility and testability.

8.4 Technologies and Frameworks

- **Programming Language:** Java 25
- **Framework:** Spring Boot 3.5.7 (Spring MVC, IoC Container)
- **Build Tool:** Maven
- **IDE:** IntelliJ IDEA
- **Front-end:** Static HTML templates with routing through controllers
- **Version Control:** Git (GitHub repository)

8.5 Architectural Benefits

- **Modularity:** Each layer performs a specific function, simplifying maintenance.

- **Flexibility:** Payment services can be added or replaced without altering core logic.
- **Testability:** Interfaces and dependency injection allow isolated unit testing.
- **Scalability:** Supports future integration with databases or additional REST microservices.

9. Implementation Details

9.1. Technologies Used

Java 25

Spring Boot 3.5.7

Maven

JUnit 5 + MockMvc

Static HTML/CSS/JS frontend

9.2. Backend Implementation

The backend is built using **Spring Boot MVC** and follows a layered structure: **Controller** → **Service** → **Model**

- **Controllers** handle REST API requests
- **Services** contain business logic
- **Models** represent data entities

```

main
├── java \ com \ codeonlineshop \ onlineshop
│   ├── dto
│   │   ├── AuthResponse.java
│   │   ├── CartitemRequest.java
│   │   ├── CheckoutRequest.java
│   │   ├── LoginRequest.java
│   │   ├── ProductRequest.java
│   │   └── RegisterRequest.java
│   └── model
│       ├── Cart.java
│       ├── Cartitem.java
│       ├── Order.java
│       ├── OrderItem.java
│       ├── OrderStatus.java
│       ├── PaymentMethod.java
│       ├── Product.java
│       ├── Role.java
│       ├── User.java
│       ├── AdminController.java
│       ├── AuthController.java
│       ├── CartController.java
│       ├── CartService.java
│       ├── HomeController.java
│       ├── OnlineShopApplication.java
│       ├── OrderController.java
│       ├── OrderService.java
│       ├── PaymentService.java
│       ├── PaymentServiceFactory.java
│       ├── PaypalPaymentService.java
│       ├── ProductController.java
│       ├── ProductService.java
│       ├── StripePaymentService.java
│       └── UserService.java

```

9.3. Authentication and Users

AuthController provides **/api/auth/register** and **/api/auth/login**

UserService validates input, hashes passwords, and stores users in memory

User model contains **id**, **username**, **passwordHash**, **role**

```
package com.codeonlineshop.onlineshop;

import com.codeonlineshop.onlineshop.dto.AuthResponse;
import com.codeonlineshop.onlineshop.dto.LoginRequest;
import com.codeonlineshop.onlineshop.dto.RegisterRequest;
import com.codeonlineshop.onlineshop.model.User;
import org.springframework.bind.annotation.PostMapping;
import org.springframework.bind.annotation.RequestBody;
import org.springframework.bind.annotation.RequestMapping;
import org.springframework.bind.annotation.RestController;

@RestController
@RequestMapping("/api/auth")
public class AuthController {
    private final UserService userService;

    public AuthController(UserService userService) {
        this.userService = userService;
    }

    /**
     * Registers a new user and returns the created user info.
     */
    @PostMapping("/register")
    public AuthResponse register(@RequestBody RegisterRequest request) {
        User user = userService.register(request);
        return new AuthResponse(user.getId(), user.getUsername(), user.getRole());
    }

    /**
     * Authenticates a user and returns their profile info for the UI.
     */
    @PostMapping("/login")
    public AuthResponse login(@RequestBody LoginRequest request) {
        User user = userService.login(request);
        return new AuthResponse(user.getId(), user.getUsername(), user.getRole());
    }
}

public User registerInternal(String username, String password, Role role) {
    long id = uuid.getId();
    User user = new User(id, username, hash(password), role);
    users.put(id, user);
    usernameIndex.put(username.toLowerCase(), id);
    return user;
}

private Role parseRole(String roleName) {
    if (roleName == null || roleName.isEmpty()) {
        return Role.CUSTOMER;
    }
    try {
        return Role.valueOf(roleName.toUpperCase());
    } catch (IllegalArgumentException e) {
        throw new RuntimeException(HttpStatus.BAD_REQUEST, "Unknown role");
    }
}

private String hash(String input) {
    try {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hashed = digest.digest(input.getBytes(StandardCharsets.UTF_8));
        StringBuilder builder = new StringBuilder();
        for (byte b : hashed) {
            builder.append(String.format("%02x", b));
        }
        return builder.toString();
    } catch (NoSuchAlgorithmException e) {
        throw new IllegalStateException("Missing SHA-256 algorithm", e);
    }
}

private boolean isValid(String value) {
    return value != null && !value.isEmpty();
}

public User loginInternal(String username, String password) {
    if (username == null || password == null) {
        throw new RuntimeException(HttpStatus.BAD_REQUEST, "Username and password are required");
    }
    Long id = usernameIndex.get(username.toLowerCase());
    if (id == null) {
        throw new RuntimeException(HttpStatus.BAD_REQUEST, "Username already exists");
    }
    User user = users.get(id);
    if (user.getPassword().equals(hash(password))) {
        return user;
    }
    throw new RuntimeException(HttpStatus.BAD_REQUEST, "Invalid credentials");
}

return user;
}
```

9.4. Product, cart, and Order Modules

Product module provides catalog and search

Cart module allows adding/removing items

Order module performs checkout and clears the cart

9.5. Payment Processing (Factory + Strategy)

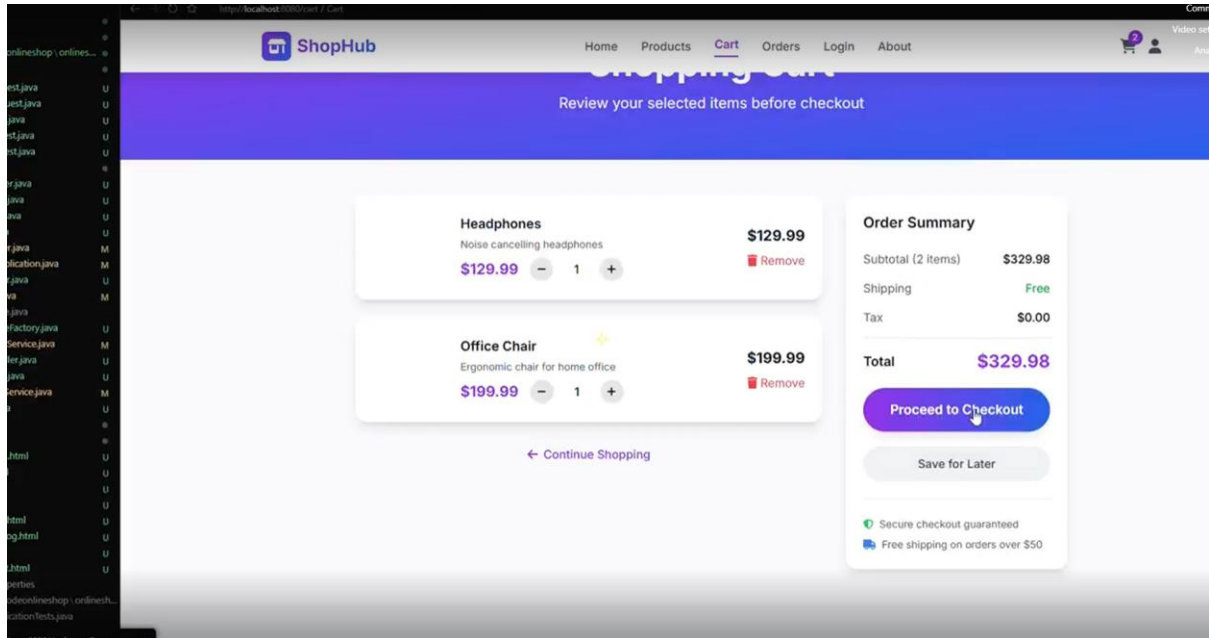
PaymentServiceFactory selects either **PaypalPaymentService** or **StripePaymentService**.

This applies the Factory and Strategy design patterns.

9.6. Fronted Implementation

Static pages are in `src/main/resources/static`.

Pages call REST APIs with JavaScript to load products, manage cart, and checkout.



10. Conclusions and Future Work

10.1 Conclusions

This project focused on the design and implementation of an **Online Shopping Information System** developed as an academic prototype using the **Waterfall software development methodology**. The objective of the project was to demonstrate a complete and structured software development process, covering all major phases including requirements analysis, system design, implementation, testing, and documentation.

The system successfully implements core e-commerce functionalities such as **user registration and authentication, product browsing, shopping cart management, checkout processing, and order history management**. The backend was developed using **Spring Boot** and follows a layered **Model-View-Controller (MVC)** architecture, ensuring a clear separation of concerns and improved maintainability.

Design patterns such as **Factory** and **Strategy** were applied in the payment processing module, allowing flexible selection of payment services and improving extensibility. Testing was performed using **JUnit 5**, **MockMvc**, and **Mockito**, covering service logic, controller endpoints, and application context loading. All defined test cases were executed successfully, confirming that the system meets its functional requirements and behaves as expected.

Overall, the project achieved its intended goals and provided valuable practical experience in applying software engineering principles, architectural patterns, and testing strategies in a web-based application.

10.2 Future Work

Although the current implementation fulfills the project requirements, several enhancements can be considered for future development:

- Integration of **real payment gateways** such as Stripe or PayPal instead of simulated payment services
- Implementation of **persistent database storage** using Spring Data JPA
- Enhancement of **security mechanisms**, including role-based authorization and token-based authentication
- Addition of **advanced product search and filtering** capabilities
- Implementation of **order delivery tracking** and notification features
- Expansion of **automated testing**, including integration and performance tests
- Deployment of the application to a **cloud-based environment** for real-world usage

These improvements would enhance the system's functionality, security, and scalability, making it suitable for production-level deployment beyond an academic prototype.

11. References

List all the documentation, articles, and online sources used during project development.

Spring Guide : Testing spring mvc with MockMvc:

<https://github.com/spring-guides/gs-testing-web>

Testing spring boot REST APIs with MockMvc:

<https://www.makareiev.com/blog/testing-spring-boot-crud-rest-api-mock-mvc/>

Example Spring boot e-commerce backend rest api:

<https://github.com/aliazmat7217/e-commerce-backend-using-rest-api>

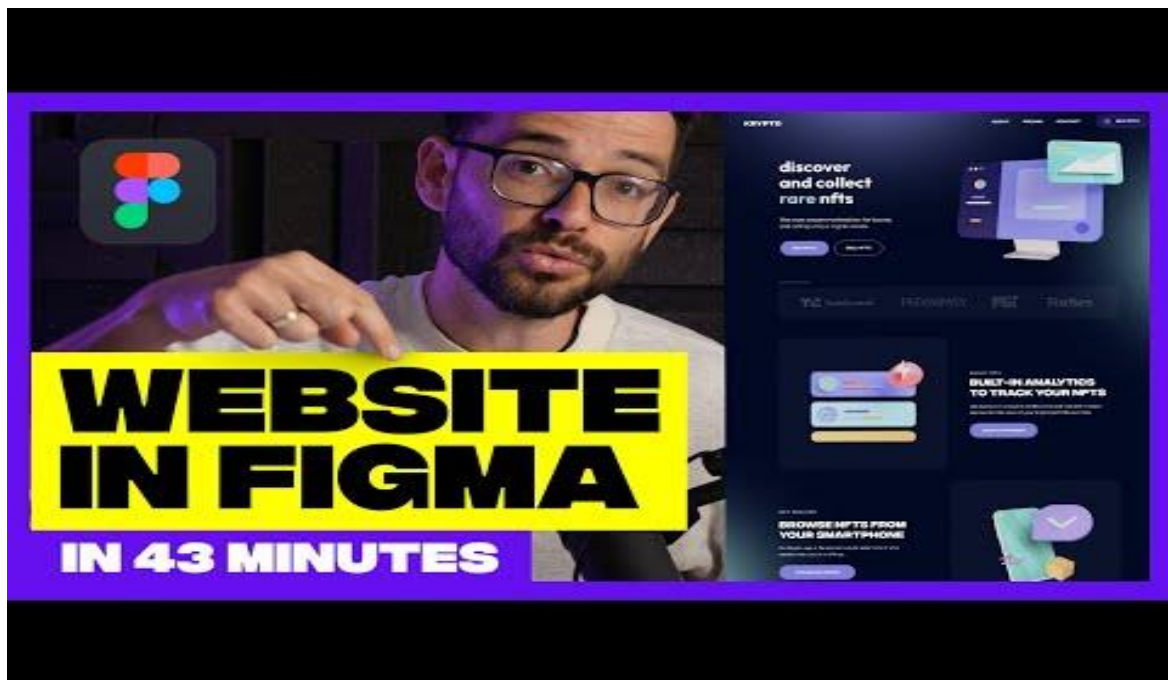
Youtube: Junit + mockmvc REST controller tests:

<https://www.youtube.com/watch?v=M8iml7gF6ZU>



Figma tutorial for beginner:

https://www.youtube.com/watch?v=HZuk6Wkx_Eg



MDN HTML basics :

https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web/HTML_basics

Spring Boot static content (serving HTML from /static)

<https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/#web.servlet.spring-mvc.static-content>

Spring official guide: testing the web layer:

<https://spring.io/guides/gs/testing-web>

